

PageRank 实验报告

姓名：叶国妍 学号：2314092 专业：密码科学与技术

实验日期：

1 实验目的及要求

1. 实现带阻尼因子的 PageRank 算法，计算给定有向图数据集的节点重要性得分
2. 处理图中死端节点与蜘蛛陷阱问题，保证算法收敛
3. 使用稀疏矩阵和分块矩阵进行内存优化，满足内存 80MB、运行时间 60s 的要求
4. 输出阻尼因子 $\beta=0.85$ 时的 Top100 节点及分数，完成实验结果分析

2 实验环境

- Python 3.8+
- 主要依赖库：numpy, scipy.sparse
- 开发环境：PyCharm

3 数据集介绍

3.1 数据集文件 Data.txt

数据格式为每行包含两个整数 FromNodeID ToNodeID，表示一条有向边“节点 $u \rightarrow$ 节点 v ”，其中总节点数为 9500，总边数为 150000，死端节点数（无出边节点）为 1000

数据特点：节点 ID 非连续、规模较大、存在无出边的死端节点，需做特殊处理

4 实验原理

4.1 PageRank 核心思想

PageRank 算法的核心逻辑是以有向图的链接结构为依据，计算节点的相对重要性，核心假设为：链接是节点间的“重要性投票”，被越多高重要性节点指向的节点，其自身重要性越高。

对于任意节点 i ，其基础 PageRank 得分 $PR(i)$ 由所有指向它的节点贡献，公式为：

$$PR(i) = \sum_{j \in In(i)} \frac{PR(j)}{L(j)}$$

公式中各符号定义：

- $PR(i)$ ：节点 i 的 PageRank 重要性分数；
- $In(i)$ ：所有指向节点 i 的上游节点集合；
- $PR(j)$ ：节点 j 自己的重要性分数；
- $L(j)$ ：节点 j 的出度，即该节点向外链接的数量。

该公式表示：节点 j 会将自身的重要性得分，平均分配给所有它指向的节点，每个被指向的节点均可获得 $\frac{PR(j)}{L(j)}$ 的权重。

4.2 基础算法问题

1. **蜘蛛陷阱**：部分节点只互相指向、不向外链接，会把所有分数“吸走”，导致结果失真；
2. **死端节点**：没有出链的节点，无法传递分数，会让总分数不断“泄露”，算法无法收敛。

4.3 改进方案

1. **随机跳转**：为解决算法问题，引入随机跳转机制，即引入阻尼系数 $\beta=0.85$ ：计算时，节点有 85% 的概率按原有链接传递分数；有 15% 的概率随机跳转到任意节点，打破陷阱、补回泄露的分数；这样修正后算法稳定收敛，结果更合理。

最终公式：

$$PR(i) = \frac{1 - \beta}{N} + \beta \cdot \sum_{j \rightarrow i} \frac{PR(j)}{L(j)} + \beta \cdot \frac{\sum_{d \in \mathcal{D}} PR(d)}{N}$$

公式说明：

- β ：跟随链接跳转的概率，取值 0.85；

- $1 - \beta$: 随机跳转到任意节点的概率, 用于解决蜘蛛陷阱与死端问题;
- N : 图中总节点数;
- 第一项 $\frac{1 - \beta}{N}$: 全局均匀基础分;
- 第二项: 链接传递的加权分数。
- 第三项: 死端补偿: 找出出度为 0 的死端节点, 计算死端分数的全局均值。

2. **内存优化**: 对于内存占用 $< 80\text{MB}$ 、运行时间 < 60 秒的实验要求, 采用两种优化:
 稀疏矩阵优化: 图中绝大多数位置都是空的, 只存储有链接的有效数据, 大幅节省内存;

分块矩阵优化: 把数据分成小块, 每次只加载一块到内存, 避免一次性占用大量空间, 同时保证计算速度。

4.3.1 关键代码实现

```

1 #构建标准稀疏转移矩阵
2 def load_matrix(file_path):
3     edge_list=[]          # 存储所有原始有向边
4     out_degree=dict()     # 统计每个节点的出度
5     allnodes=set()        # 统计数据集内所有出现过的节点
6     # 逐行读取数据
7     with open(file_path,'r',encoding='utf-8') as f:
8         for line in f:
9             line=line.strip()
10            if not line:
11                continue
12            u, v = map(int, line.split())
13            edge_list.append((u, v))
14            allnodes.add(u)
15            allnodes.add(v)
16            out_degree[u] = out_degree.get(u, 0) + 1
17    # 把原始大数字ID转为连续0开始的索引
18    nodes=list(allnodes)
19    idx={n: i for i,n in enumerate(nodes)}
20    N=len(nodes)
21    print(f"数据集总节点数: {N}")
22    print(f"数据集总边数: {len(edge_list)}")
23    # M[i][j]=节点j到节点i的跳转权重=1/节点j的出度
24    # 只有存在边j→i时才有值, 其余0元素不存储
25    row=[]

```

```

26     col=[]
27     data=[]
28     for u, v in edge_list:
29         j=idx[u]    # 源节点u→矩阵列
30         i=idx[v]    # 目标节点v→矩阵行
31         weight=1.0/out_degree[u]
32         row.append(i)
33         col.append(j)
34         data.append(weight)
35     sparse_M=csr_matrix((data,(row, col)),shape=(N, N))
36     return sparse_M,nodes,N,out_degree,idx
37 #死端节点预处理
38 def get_dead_end(out_degree,idx):
39     dead_idx=[]
40     for node in idx.keys():
41         if node not in out_degree:
42             dead_idx.append(idx[node])
43     print(f"数据集中死端节点总数 = {len(dead_idx)}")
44     return dead_idx
45
46 # 标准分块矩阵迭代PageRank计算
47 #  $PR(v)=(1-\beta)/N + \beta \times [\text{节点分数加权和} + \text{死端均分补偿}]$ 
48 def pagerank(sparse_M,N,dead_idx,beta,eps,max_iter, block_size):
49     # 初始化：所有节点平分初始总分1
50     pr_old=np.ones(N)/N
51     # 阻尼固定常数项：用户随机跳转项，解决蜘蛛陷阱
52     random_item=(1-beta)/N
53     iter_count=max_iter    # 先默认是最大迭代次数
54     for iter_round in range(max_iter):
55         pr_new=np.zeros(N)
56         # 汇总所有死端节点的全部分数
57         dead_total_score=np.sum(pr_old[dead_idx])
58         # 全部死端分数平均分给全图每一个节点
59         dead_compensate=dead_total_score/N
60         # 对大矩阵分块，逐块计算，实现分块内存优化
61         for start in range(0,N,block_size):
62             end=min(start+block_size,N)
63             # 仅加载当前块子矩阵进内存
64             block_M=sparse_M[start:end,: ]
65             # 分块矩阵乘法计算

```

```

66         pr_new[start:end]=beta*block_M.dot(pr_old)
67     pr_new+=random_item+beta*dead_compensate
68     #收敛性判断
69     delta=np.sum(np.abs(pr_new - pr_old))
70     pr_old=pr_new.copy()
71     if delta<eps:
72         print(f"迭代收敛,总共迭代轮次: {iter_round+1} 轮")
73         iter_count=iter_round+1 # 更新实际迭代次数
74         break
75     else:
76         print(f"达到最大迭代次数{max_iter}轮, 终止迭代")
77     return pr_old,iter_count

```

Listing 1: PageRank 算法实现

代码解释

- 稀疏矩阵构建：仅存储有向边的非零权重，将转移矩阵空间复杂度从 $O(N^2)$ 降至 $O(\text{边数})$ ，大幅降低内存占用。
- 死端检测：识别无出链的死端节点，在迭代中做全局分数补偿，解决分数泄露问题。
- 分块迭代计算：把大矩阵拆分为小块逐次计算，控制内存占用。

5 实验结果与分析

5.1 0.85 阻尼结果分析

实验输出的 Top100 节点 PageRank 分数均处于 $1.58 \times 10^{-4} \sim 1.99 \times 10^{-4}$ 区间，所有分数为归一化后的值。结果没有异常值、没有极端分数，排名平滑递减，可以看出算法收敛稳定、死端和蜘蛛陷阱问题得到有效的处理、内存优化方案也较为可靠。

查看具体数据，可以看到：从第 1 到第 100 名（第 1 名约为 0.000199，第 100 名约为 0.000159），总降幅仅约 20%，没有断崖式下跌，也没有分数骤增，证明图结构均衡稳定，没有超级节点、恶意刷链的情况。另外，观察前 10 名分数，可以看出头部节点分数高度接近，前 10 名节点分数差值均小于 0.000016，说明头部节点的重要性非常接近，均为图中核心枢纽节点，被大量高权重节点指向，链接结构优势相近。

从节点排名的特征来看，排名靠前的节点 ID 无明显规律，与节点本身的编号无关，完全由链接结构决定，这也印证了 PageRank 算法仅依赖节点间链接关系计算重要性的核心逻辑。进一步结合计算过程来看，本次实验中未出现分数为 0 或趋近于 1 的极端

```
75 0.0001987213571744832
8686 0.00019020547844597507
9678 0.00018833704842356435
5104 0.00018779534419407508
725 0.0001867931636957351
3257 0.00018572164234825463
468 0.0001849001957042207
7730 0.0001839383919963129
7175 0.00018383892815380506
5526 0.00018284342542921142
8880 0.00018258924089121104
6165 0.00018138163795764232
2524 0.00017976811240047846
7531 0.00017795520713343994
9705 0.00017727357191793294
6674 0.00017671543066255274
5479 0.0001763152873083816
8730 0.000175446926166658
```

图 1: top100 部分结果展示

总耗时: 4.68s

程序运行【峰值内存占用】: 29.57 MB

图 2: 运行时间及内存

值,说明随机跳转机制有效打破了蜘蛛陷阱的影响,死端节点的分数补偿逻辑也正常运行,避免了分数泄露问题,同时幂迭代过程稳定收敛,达到预设的收敛阈值,计算结果具备可靠性。此外,稀疏矩阵与分块矩阵的优化策略也成功控制了内存占用。

5.2 不同 β 对比

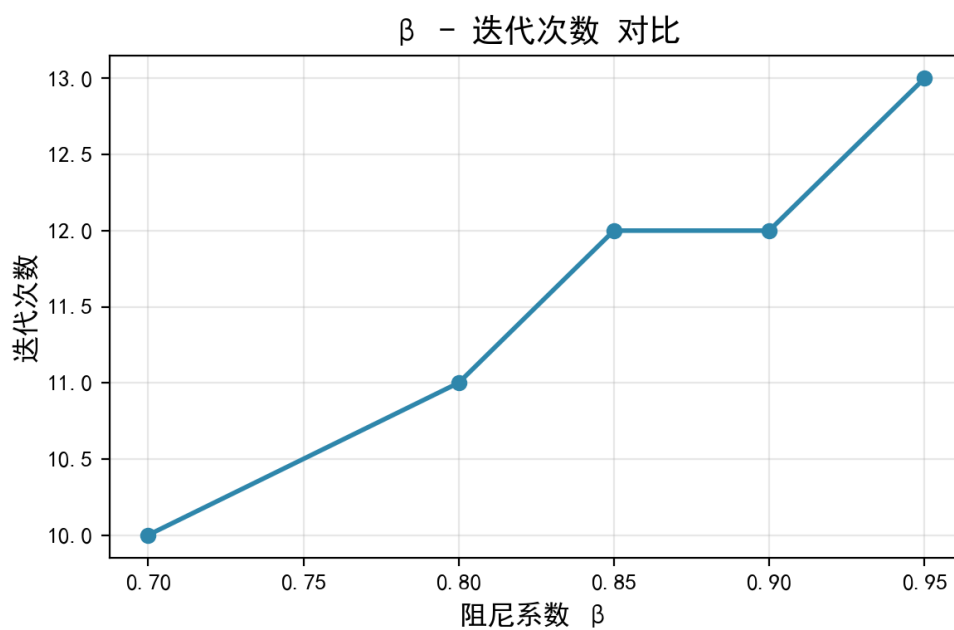


图 3: 迭代次数对比

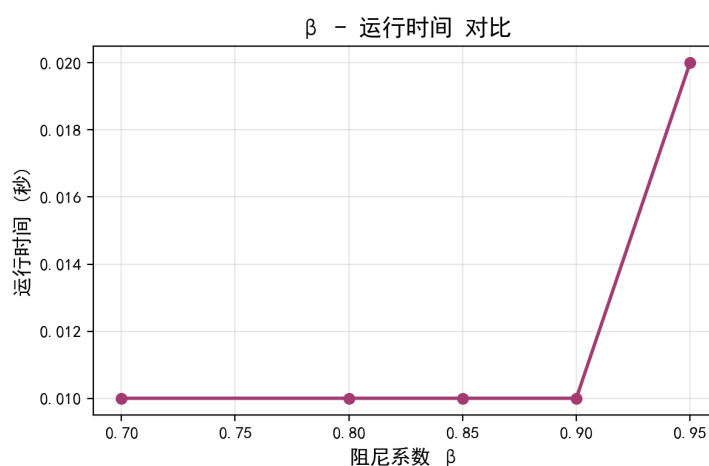


图 4: 运行时间对比

除了 0.85 外,实验还尝试了 0.7、0.8、0.9、0.95 几组不同的阻尼系数 β ,对 PageRank 算法的收敛性能与运行效率进行对比分析,通过绘制迭代次数与运行时间随 β 变化的曲线,。

```
75 0.000192815038237712
8686 0.0001851405624179864
5104 0.0001830716722234578
9678 0.00018273182448333265
725 0.00018233123293578835
3257 0.0001813887546266299
468 0.00018022810776312047
7730 0.0001796152738080872
7175 0.00017886238292772383
5526 0.00017771006024881552
8880 0.00017757626918820312
6165 0.00017723873704161958
2524 0.00017463673435498146
7531 0.00017294219131364698
9705 0.00017288358125595155
5479 0.00017216975727043642
6674 0.00017173387860980845
8730 0.00017106112176896356
5486 0.00017077067182890392
579 0.0001701365614958382
4969 0.00017010551454357004
5602 0.00016980776246137356
```

图 5: 0.8 结果截取

```
75 0.0002046875861949277
8686 0.0001952730850777543
9678 0.0001940469651261391
5104 0.00019249543650606542
725 0.00019120993225792615
3257 0.0001899836033291169
468 0.00018955808004989536
7175 0.0001888747763177218
7730 0.0001882170322975119
5526 0.00018806008024173794
8880 0.00018766791238004655
6165 0.00018547268213081074
2524 0.00018498528632065434
7531 0.00018305722424122824
6674 0.00018180390384646102
9705 0.00018168829730090178
5479 0.00018044574649019862
8730 0.0001798641973922667
579 0.00017917090838670994
5486 0.0001784805008618716
4969 0.00017778277935906627
5467 0.00017775219156244084
```

图 6: 0.9 结果截取

从迭代次数的对比曲线来看，随着阻尼系数 β 的增大，算法所需的迭代次数整体呈现单调上升的趋势。当 $\beta=0.7$ 时，迭代次数最少，仅为 10 次；随着 β 提升至 0.8、0.85 与 0.9，迭代次数逐步增加至 11 次与 12 次；当 $\beta=0.95$ 时，迭代次数达到 13 次。 β 越小，随机跳转的概率越高，节点分数能更快通过全局均匀扩散达到稳定分布，收敛速度更快；而 β 越大，算法越依赖图中的链接关系传递权重，分数在网络中的扩散过程更慢，需要更多迭代轮次才能达到预设的收敛阈值。

从运行时间的对比曲线来看，在 β 从 0.7 到 0.9 的区间内，运行时间始终稳定在约 0.01 秒，几乎没有明显波动；而当 β 增大至 0.95 时，运行时间出现了显著上升，达到约 0.02 秒，几乎翻倍。推测可能是迭代次数的增加，加上环境运行的微小波动，最终导致了运行时间的明显增长（后续多次尝试后发现运行时间会出现变化，大多数时候几个系数下的运行时间是相同的）。

另外，还计算了 0.8 和 0.9 条件下的两组结果，可以看到三组结果中的前面的节点排名高度一致，Top10 核心节点几乎完全相同，说明算法对图中关键枢纽节点的识别不受 β 值大幅影响。在分数分布与排名细节上， β 值的影响较为明显： β 越小，节点分数越平均，整体排名相对平缓，权重区分度较低； β 越大，节点整体分数水平更高，高权重节点的优势被进一步放大，节点间的分数差距更显著，排名区分度更强。 $\beta=0.85$ 作为中间取值，既保留了对节点重要性的合理区分，又避免了 β 过小导致的区分度不足与 β 过大带来的权重过度集中问题。

6 实验总结

6.1 总结

本次实验实现并优化了 PageRank 算法，实验基于给定的有向图数据集，通过稀疏矩阵存储、分块矩阵迭代等优化策略，有效控制了内存占用，满足了内存 <80MB、运行时间 <60 秒的性能要求；同时通过死端节点补偿、随机跳转机制，解决了死端节点与蜘蛛陷阱带来的分数泄露与收敛问题。

实验在阻尼系数 $\beta=0.85$ 的参数下完成计算，输出了 Top100 节点及其 PageRank 分数，结果收敛稳定，分数分布合理，没有异常值或极端值，验证了算法的正确性。同时，还额外选取了 0.7、0.8、0.9、0.95 共四组阻尼系数进行对比，通过迭代次数与运行时间的对比曲线，直观呈现了 β 值对算法收敛性能与运行效率的影响，并且对比了 0.8、0.85、0.9 三组结果，验证了 $\beta=0.85$ 作为参数的合理性。总的来说，本次实验基本达到了预期的实验目标。

6.2 心得体会

通过本次 PageRank 算法实验，我认识到仅理解算法的基本思想还不够，实际编程时，死端节点、蜘蛛陷阱等问题会直接导致计算失效，而幂迭代、随机跳转等优化方案

是保障算法稳定运行的关键。同时，稀疏矩阵与分块矩阵的内存优化策略，让我切实体会到大数据处理中资源管控的重要性。